

Path 2: Cyber Security 101 > Module 5: Networking > Room 6: Tcpdump The Basics

<https://tryhackme.com/room/tcpdump>

Tcpdump: The Basics

Learn how to use Tcpdump to save, filter, and display packets.

>> Task 1: Introduction

The main challenge when studying networking protocols is that we don't get a chance to see the protocol "conversations" taking place. All the technical complexities are hidden behind friendly and elegant user interfaces. You access resources on your local network without ever seeing an ARP query. Similarly, you would access Internet services for years without seeing a single three-way handshake till you check a networking book or inspect a network traffic capture. The best study aid would be capturing network traffic and taking a closer look at the various protocols; this helps us better understand how networks work.

This room introduces some basic command-line arguments for using Tcpdump. The Tcpdump tool and its **libpcap** library are written in C and C++ and were released for Unix-like systems in the late 1980s or early 1990s. Consequently, they are very stable and offer optimal speed. The **libpcap** library is the foundation for various other networking tools today. Moreover, it was ported to MS Windows as **winpcap**.

♦ Learning Objectives:

This room aims to provide you with the basics necessary to use **tcpdump**. In particular, you will learn how to:

- Capture packets and save them to a file
- Set filters on captured packets
- Control how captured packets are displayed

Q1) What is the name of the library that is associated with **tcpdump**?

Answer: libpcap

>> Task 2: Basic Packet Capture

You can run `tcpdump` without providing any arguments; however, this is only useful to test that you have it installed! In any real scenario, we must be specific about what to listen to, where to write, and how to display the packets.

Command: `sudo tcpdump`

- ◆ Specify the Network Interface:

The first thing to decide is which network interface to listen to using `-i INTERFACE`. You can choose to listen on all available interfaces using `-i any`; alternatively, you can specify an interface you want to listen on, such as `-i eth0`.

A command such as `ip address show` (or merely `ip a s`) would list the available network interfaces. In the terminal below, we see one network card, `ens5`, in addition to the loopback address.

Command: `sudo tcpdump -i <InterFace>`

- ◆ Save the Captured Packets:

In many cases, you should check the captured packets again later. This can be achieved by saving to a file using `-w FILE`. The file extension is most commonly set to `.pcap`. The saved packets can be inspected later using another program, such as Wireshark. You won't see the packets scrolling when you choose the `-w` option.

Command: `sudo tcpdump -w <file.pcap>`

- ◆ Read Captured Packets from a File:

You can use `Tcpdump` to read packets from a file by using `-r FILE`. This is very useful for learning about protocol behaviour. You can capture network traffic over a suitable time frame to inspect a specific protocol, then read the captured file while applying filters to display the packets you are interested in. Furthermore, it might be a packet capture file that contains a network attack that took place, and you inspect it to analyze the attack.

Command: `sudo tcpdump -r <file.pcap>`

- ◆ Limit the Number of Captured Packets:

You can specify the number of packets to capture by specifying the count using `-c COUNT`. Without specifying a count, the packet capture will continue till you interrupt

it, for example, by pressing CTRL-C. Depending on your goal, you only need a limited number of packets.

Command: `sudo tcpdump -c <CountNum>`

- ◆ Don't Resolve IP Addresses and Port Numbers:

Tcpdump will resolve IP addresses and print friendly domain names where possible. To avoid making such DNS lookups, you can use the `-n` argument. Similarly, if you don't want port numbers to be resolved, such as **80** being resolved to **http**, you can use the `-nn` to stop both DNS and port number lookups. Consider the following example shown in the terminal below. We captured and displayed five packets without resolving the IP addresses.

Command: `sudo tcpdump -i eth0 -c 5 -n` (Don't resolve IP's to Domain-names)
Command: `sudo tcpdump -i eth0 -c 5 -nn` (Don't resolve both IP to Domain-names & Protocol Numbers to Protocol)

- ◆ Produce (More) Verbose Output:

If you want to print more details about the packets, you can use `-v` to produce a slightly more verbose output. According to the Tcpdump manual page ([man tcpdump](#)), the addition of `-v` will print "the time to live, identification, total length and options in an IP packet" among other checks. The `-vv` will produce more verbose output; the `-vvv` will provide even more verbosity; check the manual page for details.

- ◆ Summary and Examples:

Command	Explanation
<code>tcpdump -i INTERFACE</code>	Captures packets on a specific network interface
<code>tcpdump -w FILE</code>	Writes captured packets to a file
<code>tcpdump -r FILE</code>	Reads captured packets from a file
<code>tcpdump -c COUNT</code>	Captures a specific number of packets
<code>tcpdump -n</code>	Don't resolve IP addresses
<code>tcpdump -nn</code>	Don't resolve IP addresses and don't resolve protocol numbers

<code>tcpdump -v</code>	Verbose display; verbosity can be increased with "-vv" and "-vvv"
-------------------------	---

Consider the following examples:

- `tcpdump -i eth0 -c 50 -v` captures and displays 50 packets by listening on the `eth0` interface, which is a wired Ethernet, and displays them verbosely.

- `tcpdump -i wlo1 -w data.pcap` captures packets by listening on the `wlo1` interface (the WiFi interface) and writes the packets to `data.pcap`. It will continue till the user interrupts the capture by pressing CTRL-C.

- `tcpdump -i any -nn` captures packets on all interfaces and displays them on screen without domain name or protocol resolution.

Q1) What option can you add to your command to display addresses only in numeric format?

Answer: -n

>> Task 3: Filtering Expressions

Although you can run **tcpdump** without providing any filtering expressions, this won't be useful. Just like in a social gathering, you don't try to listen to everyone at the same time; you would rather give your attention to a specific person or conversation. Considering the number of packets seen by our network card, it is impossible to see everything at once; we need to be specific and capture what we are interested in inspecting.

♦ Filtering by Host:

Let's say you are only interested in IP packets exchanged with your network printer or a specific game server. You can easily limit the captured packets to this host using **host IP** or **host HOSTNAME**. In the terminal below, we capture all the packets exchanged with **example.com** and save them to **http.pcap**. It is important to note that capturing packets requires you to be logged-in as **root** or to use **sudo**.

Command: **sudo tcpdump host example.com -w http.pcap**

Command: **sudo tcpdump host 250.156.25.20 -w http.pcap**

If you want to limit the packets to those from a particular source IP address or hostname, you must use **src host IP** or **src host HOSTNAME**. Similarly, you can limit packets to those sent to a specific destination using **dst host IP** or **dst host HOSTNAME**.

♦ Filtering by Port:

If you want to capture all DNS traffic, you can limit the captured packets to those on **port 53**. Remember that DNS uses UDP and TCP ports 53 by default. In the following example, we can see all the DNS queries read by our network card. The terminal below shows two DNS queries: the first query requests the IPv4 address used by example.org, while the second requests the IPv6 address associated with example.org.

Command: **sudo tcpdump -i ens5 port 53 -n**

♦ Filtering by Protocol:

The final type of filtering we will cover is filtering by protocol. You can limit your packet capture to a specific protocol; examples include: **ip**, **ip6**, **udp**, **tcp**, and **icmp**. In the example below, we limit our packet capture to ICMP packets. We can see an ICMP echo request and reply, which is a possible indication that someone is running the **ping** command. There is also an ICMP time exceeded; this might be due to running the **traceroute** command (as explained in the **Networking Essentials** room).

Command: `sudo tcpdump -i ens5 icmp -n`

♦ Logical Operators:

Three logical operators that can be handy:

- **and**: Captures packets where both conditions are true. For example, `tcpdump host 1.1.1.1 and tcp` captures **tcp** traffic with **host 1.1.1.1**.

- **or**: Captures packets when either one of the conditions is true. For instance, `tcpdump udp or icmp` captures UDP or ICMP traffic.

- **not**: Captures packets when the condition is not true. For example, `tcpdump not tcp` captures all packets except TCP segments; we expect to find UDP, ICMP, and ARP packets among the results.

♦ Summary and Examples:

Command	Explanation
<code>tcpdump host IP</code> or <code>tcpdump host HOSTNAME</code>	Filters packets by IP address or hostname
<code>tcpdump src host IP</code> or	Filters packets by a specific source host
<code>tcpdump dst host IP</code>	Filters packets by a specific destination host
<code>tcpdump port PORT_NUMBER</code>	Filters packets by port number
<code>tcpdump src port PORT_NUMBER</code>	Filters packets by the specified source port number
<code>tcpdump dst port PORT_NUMBER</code>	Filters packets by the specified destination port number
<code>tcpdump PROTOCOL</code>	Filters packets by protocol; examples include ip , ip6 , and icmp

Consider the following examples:

- `tcpdump -i any tcp port 22` listens on all interfaces and captures **tcp** packets to or from **port 22**, i.e., SSH traffic.

- `tcpdump -i wlo1 udp port 123` listens on the WiFi network card and filters **udp** traffic to **port 123**, the Network Time Protocol (NTP).

- `tcpdump -i eth0 host example.com and tcp port 443 -w https.pcap` will listen on `eth0`, the wired Ethernet interface and filter traffic exchanged with `example.com` that uses `tcp` and `port 443`. In other words, this command is filtering HTTPS traffic related to `example.com`.

For the questions from this task, we will read captured packets from the `traffic.pcap` file. As mentioned earlier, we use `-r FILE` to read from a packet capture file. To test this, try `tcpdump -r traffic.pcap -c 5 -n`; it should display the first five packets in the file without looking up the IP addresses.

Remember that you can count the lines by piping the output via the `wc` command. In the terminal below, we can see that we have 910 packets with the source IP address set to `192.168.124.1`. Please note that we add `-n` to avoid unnecessary delays in attempting to resolve IP addresses. In the example below, we didn't use `sudo` as reading from a packet capture file does not require `root` privileges.

Command: `tcpdump -r traffic.pcap src host 192.168.124.1 -n | wc`

Q1) How many packets in `traffic.pcap` use the ICMP protocol?

Answer: 26

Command: `tcpdump -r traffic.pcap icmp -n | wc`

Q2) What is the IP address of the host that asked for the MAC address of 192.168.124.137?

Answer: 192.168.124.148

Command: `tcpdump -r traffic.pcap arp -n`

Q3) What hostname (subdomain) appears in the first DNS query?

Answer: mirrors.rockylinux.org

Command: `tcpdump -r traffic.pcap port 53 -n`

>> Task 4: Advanced Filtering

There are many more ways to filter packets. After all, in any real-life situation, we would need to filter through thousands or even millions of packets. It is indispensable to be able to express the exact packets to display. For example, we can limit the displayed packets to those smaller or larger than a certain length:

- **greater LENGTH**: Filters packets that have a length greater than or equal to the specified length
- **less LENGTH**: Filters packets that have a length less than or equal to the specified length

We recommend you check the **pcap-filter** manual page by issuing the command **man pcap-filter**; however, for the purposes of this room, we will focus on one advanced option that allows you to filter packets based on the TCP flags. Understanding the TCP flags will make it easy to build on this knowledge and master more advanced filtering techniques.

♦ Binary Operations:

Before proceeding, it is worth visiting binary operations. A binary operation works on bits, i.e., zeroes and ones. An operation takes one or two bits and returns one bit. Let's explain in more depth and consider the following three binary operations: **&**, **|**, and **!**.

& (And) takes two bits and returns 0 unless both inputs are 1, as shown in the table below.

Input 1	Input 2	Input 1 & Input 2
0	0	0
0	1	0
1	0	0
1	1	1

| (Or) takes two bits and returns 1 unless both inputs are 0. This is shown in the table below.

Input 1	Input 2	Input 1 Input 2
0	0	0
0	1	1
1	0	1
1	1	1

! (Not) takes one bit and inverts it; an input of 1 gives 0, and an input of 0 gives 1, as shown in the table below.

Input 1	! Input 1
0	1
1	0

♦ Header Bytes:

The purpose of this section is to be able to filter packets based on the contents of a header byte. Consider the following protocols: ARP, Ethernet, ICMP, IP, TCP, and UDP. These are just a few networking protocols we have studied. How can we tell Tcpcap to filter packets based on the contents of protocol header bytes? (We will not go into details about the headers of each protocol as this is beyond the scope of this room; instead, we will focus on TCP flags.)

Using pcap-filter, Tcpcap allows you to refer to the contents of any byte in the header using the following syntax **proto[expr:size]**, where:

- **proto** refers to the protocol. For example, **arp**, **ether**, **icmp**, **ip**, **ip6**, **tcp**, and **udp** refer to ARP, Ethernet, ICMP, IPv4, IPv6, TCP, and UDP respectively.

- **expr** indicates the byte offset, where **0** refers to the first byte.

- **size** indicates the number of bytes that interest us, which can be one, two, or four. It is optional and is one by default.

To better understand this, consider the following two examples from the pcap-filter manual page (and don't worry if you find them difficult):

- **ether[0] & 1 != 0** takes the first byte in the Ethernet header and the decimal number 1 (i.e., **0000 0001** in binary) and applies the **&** (the And binary operation). It will return true if the result is not equal to the number 0 (i.e., **0000 0000**). The purpose of this filter is to show packets sent to a multicast address. A multicast Ethernet address is a particular address that identifies a group of devices intended to receive the same data.

- **ip[0] & 0xf != 5** takes the first byte in the IP header and compares it with the hexadecimal number F (i.e., **0000 1111** in binary). It will return true if the result is not equal to the (decimal) number 5 (i.e., **0000 0101** in binary). The purpose of this filter is to catch all IP packets with options.

Don't worry if you find the above two examples complex. We included them so you know what you can achieve with this; however, fully understanding the above examples is not necessary to finish this task. Instead, we will focus on filtering TCP packets based on the set TCP flags.

You can use **tcp[tcpflags]** to refer to the TCP flags field. The following TCP flags are available to compare with:

- **tcp-syn** TCP SYN (Synchronize)
- **tcp-ack** TCP ACK (Acknowledge)
- **tcp-fin** TCP FIN (Finish)
- **tcp-rst** TCP RST (Reset)
- **tcp-push** TCP Push

Based on the above, we can write:

- **tcpdump "tcp[tcpflags] == tcp-syn"** to capture TCP packets with only the SYN (Synchronize) flag set, while all the other flags are unset.

- **tcpdump "tcp[tcpflags] & tcp-syn != 0"** to capture TCP packets with at least the SYN (Synchronize) flag set.

- **tcpdump "tcp[tcpflags] & (tcp-syn|tcp-ack) != 0"** to capture TCP packets with at least the SYN (Synchronize) or ACK (Acknowledge) flags set.

You can write your own filter depending on what you are looking for.

Q1) How many packets have only the TCP Reset (RST) flag set?

Answer: 57

Command: `tcpdump -r traffic.pcap "tcp[tcpflags] == tcp-rst" | wc`

Q2) What is the IP address of the host that sent packets larger than 15000 bytes?

Answer: 185.117.80.53

Command: `tcpdump -r traffic.pcap greater 15000 -n`

>> Task 5: Displaying Packets

Tcpdump is a rich program with many options to customize how the packets are printed and displayed. We have selected to cover the following five options:

- q: Quick output; print brief packet information
- e: Print the link-level header
- A: Show packet data in ASCII
- xx: Show packet data in hexadecimal format, referred to as hex
- X: Show packet headers and data in hex and ASCII

- ◆ Brief Packet Information:

If you prefer shorter output lines, you can opt for “**quick**” output with -q. The following example shows the timestamp, along with the source and destination IP addresses and source and destination port numbers.

Command: `tcpdump -r traffic.pcap -q`

- ◆ Displaying Link-Level Header:

If you are on an Ethernet or WiFi network and want to include the MAC addresses in Tcpdump output, all you need to do is to add -e. This is convenient when you are learning how specific protocols, such as ARP and DHCP function. It can also help you track the source of any unusual packets on your network.

Command: `tcpdump -r traffic.pcap -e`

- ◆ Displaying Packets as ASCII:

ASCII stands for American Standard Code for Information Interchange; ASCII codes represent text. In other words, you can expect -A to display all the bytes mapped to English letters, numbers, and symbols.

Command: `tcpdump -r traffic.pcap -A`

- ◆ Displaying Packets in Hexadecimal Format:

ASCII format works well when the packet contents are plain-text English. It won't work if the contents have undergone encryption or even compression. Furthermore, it won't work for languages that don't use the English alphabet. Hence, we need another way to display the packet contents regardless of format. Being 8 bits, any octet can be displayed as two hexadecimal digits. (Each hexadecimal digit represents 4 bits.) To display the packets in hexadecimal format, we must add -xx.

Command: `tcpdump -r traffic.pcap -xx`

♦ Best of Both Worlds:

If you would like to display the captured packets in hexadecimal and ASCII formats, Tcpcap makes it easy with the `-X` option.

Command: `tcpdump -r traffic.pcap -X`

♦ Summary and Examples:

Command	Explanation
<code>tcpdump -q</code>	Quick and quite: brief packet information
<code>tcpdump -e</code>	Include MAC addresses
<code>tcpdump -A</code>	Print packets as ASCII encoding
<code>tcpdump -xx</code>	Display packets in hexadecimal format
<code>tcpdump -X</code>	Show packets in both hexadecimal and ASCII formats

Q1) What is the MAC address of the host that sent an ARP request?

Answer: 52:54:00:7c:d3:5b

Command: `tcpdump -r traffic.pcap arp -e`